

METODOLOGIA DE ENGENHARIA · HUMANO + IA

Maestro

Compêndio de Governança

Constituição · Modelo operacional · Handbook · Decisões.



PLATAFORMA GHDARU TECNOLOGIA · GHDARU/MAESTRO
VISÃO GERAL – TODOS OS DOCUMENTOS DE GOVERNANÇA EM UM VOLUME
GERADO DA FONTE VERSIONADA

Sumário

Parte I Princípios (constituição da metodologia)

Parte II Modelo operacional (a regra vigente)

Parte III Handbook — fundamentos por elemento (12 capítulos)

Apêndice Decisões de arquitetura (ADRs 0004-0007)

PARTE I

Princípios

*A constituição da metodologia — os inegociáveis
do Maestro.*

Princípios do Maestro (constituição da metodologia)

Fonte de verdade das convenções do **Maestro**. Prevalece sobre qualquer outra prática deste repositório. **Todo agente e humano DEVE ler este documento antes de qualquer trabalho**. Emendas via ADR + bump de versão.

Versão: 1.0.0 · **Ratificada:** 2026-07-22

Maestro é a metodologia de **1 humano regendo N agentes de IA**: a especificação é a fonte de verdade, os agentes executam, o humano decide/aprova/verifica. Estes são os princípios inegociáveis; o **modelo operacional** (`modelo-operacional.md`) os operacionaliza e o **handbook** (`../handbook/`) os fundamenta.

Princípios

I. Spec-Driven (a spec é a fonte de verdade)

Nenhum código nasce sem especificação. A spec é o **input que gera** o código, não a sua descrição — por isso não apodrece. Fluxo `specify → clarify → plan → tasks → implement`. Mudança de escopo volta à spec antes de virar código.

II. Orquestração humano-governada (1 rege N)

Papéis são **modos de trabalho** numa matriz humano×agentes (RACI). Delega-se **Responsible/Consulted/Informed** a agentes; o **Accountable é sempre humano** e responde **pela política, gates e critérios — não por cada item**. Verificação por **agente independente em contexto fresco** (quem executa não verifica).

III. Reversibilidade e gates proporcionais ao risco (NON-NEGOTIABLE)

O gate humano escala com **irreversibilidade × impacto** (taxonomia: leitura → ...→ irreversível → bloqueado). O que torna o irreversível seguro de delegar é a **reversibilidade engenheirada** (backup, dry-run, staging, soft-delete), que **rebaixa a classe de risco**. Política declarativa `allow / deny / ask`; autorização fora do LLM.

IV. Test-First e DoD verificável ("prove, não declare")

Testes nascem com (ou antes) o código. "Pronto" é **verificável autonomamente** (pass/fail que o agente gera e um hook confere); o trabalho é **converter julgamento em check**. Verde local ≠ certo global — coerência global e "a coisa certa" ficam com o humano. CI e testes de arquitetura (fitness functions) são gate desde a fundação.

V. Economia de contexto e corte por fronteira

A janela de contexto é finita e degrada ao encher: preserve o **contexto integrador (a spec)**, descarte o ruído. Paralelize **por bounded context** (DDD/hexagonal) — bons cortes tornam a orquestração segura. Menor autonomia que resolve (workflow antes de agent).

VI. Artefatos vivos e rastreabilidade

Um artefato só existe se é **input consumido com forcing function** (ou imutável, como o ADR). Não duplicar função já servida. A rastreabilidade **spec ↔ PR ↔ testes ↔ journey** é a memória durável do projeto — emerge do workflow, sem ferramenta pesada.

VII. Governança leve que aprende (YAGNI)

A governança **aprende sem inchar**: núcleo firme (esta constituição) + periferia evoluível (modelo/handbook, versão própria) + memória append-only (ADRs) + **retro → regra versionada**. **YAGNI** poda o que não paga. Complexidade além do necessário é justificada por escrito ou removida.

Governança

Esta constituição prevalece; emendas sobem versão semântica (MAJOR: remoção/redefinição; MINOR: novo princípio/expansão; PATCH: clarificação) e são registradas em ADR. Toda decisão material de metodologia vira ADR.

Nota de linhagem (migração)

Os documentos migrados de `ghdaru` / `flowbuilder` citam "a Constituição" e "Princípio IV/V/ VII" referindo-se à constituição **da plataforma** de origem. No Maestro, esses papéis correspondem aos princípios acima (mapa aproximado: IV→III+IV, V→IV, VII→II+VI). A reescrita fina dessas referências para apontar a este documento é **follow-up** registrado no ADR 0007.

PARTE II

Modelo operacional

*A regra vigente: papéis, cerimônias, artefatos,
raias, gates, DoR/DoD, enforcement.*

Modelo Operacional — Papéis, Cerimônias, Entregáveis e Artefatos

Como o trabalho é conduzido na Plataforma GHDaru Tecnologia num contexto de **um humano orquestrando múltiplos agentes de IA**. Este documento define QUEM faz O QUÊ (papéis/responsabilidades), QUANDO (cerimônias/cadência) e O QUE se produz (entregáveis/artefatos com gate).

Status: Ativo · **Versão:** 1.2.0 · **Data:** 2026-07-22 · **Fundamentação:**

`docs/research/resultado-pesquisa-praticas-desenvolvimento-avaliacao.md` · **Decisões:**

`docs/adr/0004-modelo-operacional.md`, `docs/adr/0005-raias-de-trabalho-e-specs-de-infra.md`,

`docs/adr/0006-enforcement-dod-changelog.md`

1. Propósito e escopo

A Constituição (`.specify/memory/constitution.md`) define os **princípios** (o que é inegociável) e o Spec Kit define o **fluxo técnico de uma feature** (`/speckit.specify → clarify → plan → tasks → implement`). Faltava a camada de cima: o **modelo operacional** — a distribuição de papéis entre humano e agentes, a cadência de trabalho e o catálogo de entregáveis com seus portões de qualidade.

Este documento **não substitui** a Constituição nem o Spec Kit; ele os **integra**. Onde houver conflito, a Constituição prevalece (Governance).

2. Princípio operacional central

Derivado da pesquisa (síntese e Princípios I, V, VII da Constituição):

IA para explorar, propor e escrever; humano para especificar, decidir e aprovar; testes, gates e revisão independente para validar.

Três regras operacionais decorrem dele:

1. **A spec é a fonte de verdade, não o código nem o prompt.** Todo trabalho nasce de uma especificação (Princípio I). O humano *dirige e refina*; o agente escreve.
2. **Quem executa não é quem verifica.** A verificação final passa por um **agente revisor em contexto fresco** (e/ou humano), nunca pelo mesmo agente que produziu o código. É o contrapeso que substitui o "segundo par de olhos" de um time.
3. **Prove, não declare.** "Pronto" exige evidência que um agente consiga gerar e um gate consiga conferir: teste verde, build limpo, lint, screenshot, journey atualizada.

3. Raias de trabalho (quanto processo cada mudança recebe)

Nem toda mudança merece uma spec completa. O **valor de uma spec escala com ambiguidade × raio de impacto × irreversibilidade** — quando os três são baixos, a spec é cerimônia de papel (YAGNI); quando algum é alto, ela se paga. Daí três raias:

RAIA	QUANDO	FLUXO	REGISTRO	DOD
Leve (delta)	bug, typo, rename, log, copy — "dá para descrever o diff numa frase"	direto: explore → code → commit (sem <code>spec.md</code> / <code>plan.md</code> / <code>tasks.md</code>)	o PR é o artefato (descreve o delta); bug exige um teste que o reproduz (falha primeiro)	reduzida: testes + fitness + revisão independente; doc viva só se tocar jornada
Plena (spec)	feature ambígua, contrato, mudança cross-feature	Spec Kit completo: specify → clarify → plan → tasks → implement	<code>specs/NNN-*/</code>	completa (§7)
Infra	infra, migração, deploy	Spec Kit completo — sempre plena , nunca leve (mesmo parecendo pequena)	<code>specs/NNN-*/</code>	completa + gates de reversibilidade (§7)

Regras da raia leve: (1) na dúvida entre leve e plena, é **plena**; (2) infra e migração **nunca** são leves — o raio e a irreversibilidade as jogam direto na raia infra; (3) a raia leve **não** afrouxa a revisão independente nem os testes — só dispensa os artefatos de planejamento (`spec` / `plan` / `tasks`).

Uma única ferramenta de spec-driven (Spec Kit). A raia leve (modelo *delta*) mora **dentro** dela, não numa segunda ferramenta. Ver §10 (OpenSpec avaliado e descartado).

4. Papéis e responsabilidades

Num time minúsculo os papéis clássicos **acumulam-se**, mas os **contrapesos** são preservados movendo o *verifica* para um agente independente e mantendo o *decide* sempre no humano. Cada "papel" é um **modo de trabalho** exercido por humano, agente, ou os dois com gate.

PAPEL (MODO)	EXERCIDO POR	RESPONSABILIDADE	GATE
Product Steward / PO	Humano	Decide o quê e por quê; prioriza; define apetite do ciclo; aprova specs e releases	Accountable de tudo
Arquiteto / Tech Lead	Humano + <code>plan-agent</code>	Decisões arquiteturais (DDD/hexagonal, contratos, ADRs); Constitution Check	Humano aprova o plan
Spec-agent (<code>/speckit.specify</code> , <code>clarify</code>)	Agente (humano decide)	Redige <code>spec.md</code> a partir da intenção; levanta ambiguidades	Humano aprova a spec
Plan-agent (<code>/speckit.plan</code>)	Agente	Redige <code>plan.md</code> , <code>ux-design.md</code> , Constitution Check	Humano aprova o plan
UX-agent	Agente + skills	Consulta design system + camada semântica; define <code>ux-design.md</code>	Objeto semântico obrigatório (P. VII)

PAPEL (MOD0)	EXERCIDO POR	RESPONSABILIDADE	GATE
Dev-agent (<code>/speckit.implement</code>)	Agente	Implementa tasks; escreve testes; diffs pequenos	Testes verdes + fitness functions
QA/SDET-agent	Agente	Cobertura feliz + falha por caso de uso; testes de contrato/arquitetura	DoD (§7)
Review-agent (revisor independente)	Agente em contexto fresco	Revisa o diff contra o plan; aponta lacunas de correção/requisito	<code>/code-review</code> , gate de merge
Security-agent	Agente	Revisão de injeção/segredos/autorização; secret scanning	Gate de segurança leve (§7)
Tech Writer-agent	Agente	Atualiza <code>docs/journeys/</code> , ADRs, changelog no mesmo PR	Documentação viva (P. VII)
Orquestrador	Humano	Sequencia agentes; <code>/clear</code> entre tarefas; decide paralelo/pipeline; para quando algo é caro de reverter	—

RACI por etapa do ciclo de vida (R = executa, A = aprova/accountable, C = consultado, I = informado):

ETAPA	EXECUTA (R)	VERIFICA (C)	APROVA (A)
Definir intenção/apetite	Humano	—	Humano
Spec (<code>spec.md</code>)	Spec-agent	Humano	Humano
Plan + Constitution Check	Plan-agent	Review-agent	Humano
UX design	UX-agent	Humano	Humano
Implementação	Dev-agent	QA-agent	Review-agent → Humano
Revisão de código	Review-agent (fresco)	Security-agent	Humano
Documentação/jornadas	Tech Writer-agent	Humano	Humano
Release/deploy	Humano + CI	Review-agent	Humano

O humano é Accountable fixo em toda linha de risco. Nenhum agente decide sozinho nada que caia nas classes de risco de alteração/exclusão/externa/irreversível (Constituição, Princípio IV) — ver §8.

5. Cerimônias e cadência

Adotamos o **esqueleto Shape Up + Kanban**, não o Scrum. Cortamos daily e sprint planning (cerimônia de papel para um solo). Trabalho em **fluxo contínuo com WIP limitado a uma feature/spec por vez**, organizado em **ciclos com appetite fixo**.

CERIMÔNIA	CADÊNCIA	QUEM	PROPÓSITO	SAÍDA
Shaping / Spec	Por feature	Humano + spec/plan-agent	Definir appetite, escopo e critérios de aceite testáveis; é o "planning" real	<code>spec.md</code> aprovada
Execução do ciclo	Contínua (apetite fixo)	Orquestrador + agentes	Implementar com escopo variável dentro do appetite; parar ao estourar o appetite, não ao "terminar tudo"	PRs mergeados
Checkpoint de ciclo	Fim de cada ciclo	Humano	Inspecionar o que entrou; decidir cooldown/próxima aposta	Decisão de próxima spec
Cooldown	Após o ciclo	Humano + agentes	Dívida técnica, manutenção, bugs pequenos (raia leve, §3), curadoria de skills	—
Retro individual	Fim de cada ciclo	Humano	O que os agentes erraram de forma recorrente → vira regra	Regra em <code>CLAUDE.md</code> /constituição/ADR

A retro é a cerimônia de maior ROI neste modelo. Cada correção recorrente que você faz num agente deve virar **instrução versionada** (`CLAUDE.md` , skill, ou emenda de constituição). O processo aprende; você não repete a mesma correção.

Appetite (Shape Up): o ciclo tem tempo fixo e escopo variável — o oposto de estimar prazo. Casa direto com YAGNI e "diffs pequenos": quando o appetite acaba, corta-se escopo, não se estende o prazo.

6. Entregáveis e artefatos

Catálogo dos artefatos do ciclo de vida, com **dono**, **gate** e **onde vive**. "Essencial" = obrigatório agora; "Fase 2/YAGNI" = observado, não adotado (§10).

ARTEFATO	DONO	ONDE VIVE	GATE	STATUS
Brief / intenção	Humano	issue/spec header	—	Essencial
Spec (<code>spec.md</code>)	Spec-agent	<code>specs/NNN-*/</code>	Humano aprova (DoR)	Essencial (raia plena)
Plan (<code>plan.md</code>) + Constitution Check	Plan-agent	<code>specs/NNN-*/</code>	Humano aprova	Essencial (raia plena)
UX design (<code>ux-design.md</code>)	UX-agent	<code>specs/NNN-*/</code>	Objeto semântico (P. VII)	Essencial (se UI)
Tasks (<code>tasks.md</code>)	Plan-agent	<code>specs/NNN-*/</code>	—	Essencial (raia plena)
Spec de infra	Humano + agente	<code>specs/NNN-*/</code>	Sempre raia plena + gates de reversibilidade (§7)	Essencial (infra/migração)

ARTEFATO	DONO	ONDE VIVE	GATE	STATUS
Código + testes	Dev-agent	apps/	Testes + fitness functions verdes	Essencial
ADR	Tech Writer-agent	docs/adr/NNNN-*.md	Decisão arquitetural registrada	Essencial (por decisão)
Journey doc	Tech Writer-agent	docs/journeys/NNN-*.md	Capturas + heurística no mesmo PR (P. VII)	Essencial (se jornada)
Runbook	Humano + agente	docs/infra/	Estratégia de rollback documentada	Essencial (infra)
Changelog / release notes	Tech Writer-agent	CHANGELOG.md	Atualizado no PR de release	A formalizar
Definition of Ready/Done	Humano	este doc (§7)	Checklist verificável	Essencial (este doc)
PR	Dev-agent	GitHub	Gate de merge (§7); na raia leve, o PR é o próprio artefato	Essencial
C4 diagrams	—	docs/architecture/	—	Fase 2 / YAGNI
RFC process pesado	—	—	—	YAGNI (ADR basta)
Painel de métricas DORA	—	—	—	Fase 2 / YAGNI

7. Definition of Ready e Definition of Done

Escritas como **checklists verificáveis** — um agente consegue satisfazê-las e um Stop-hook/ `/goal` consegue conferi-las. A DoR aplica-se à **raia plena** (§3); a raia leve entra direto na execução com a **DoD reduzida** indicada abaixo.

Definition of Ready (uma spec pode entrar em execução? — raia plena)

- [] `spec.md` descreve **o quê e por quê** com critérios de aceite **testáveis**.
- [] Ambiguidades resolvidas (`/speckit.clarify`) ou registradas.
- [] `plan.md` passou pelo **Constitution Check** (violação → justificada ou removida).
- [] `ux-design.md` declara papéis/objetos semânticos consumidos/introduzidos (se UI).
- [] **Apetite** definido (tempo fixo do ciclo).

Definition of Done (uma feature está pronta?)

- [] Testes verdes: unitário de domínio + contrato por porta + integração por rota.
- [] **Fitness functions** (regras de dependência DDD/hexagonal) verdes no CI (P. V).
- [] Lint + typecheck limpos.
- [] **Revisão independente** (`/code-review` em contexto fresco) sem lacunas de correção/requisito abertas.

- [] **Gate de segurança leve:** secret scanning limpo; revisão de injeção/autorização onde aplicável (P. IV).
- [] **Documentação viva atualizada no mesmo PR:** journey (capturas + heurística), ADR se houve decisão, changelog.
- [] **Rastreabilidade** registrada: spec NNN ↔ PR ↔ testes ↔ journey (§9).
- [] Evidência anexada (output de teste/build/screenshot) — "prove, não declare".

DoD reduzida (raia leve, §3): valem os itens de testes, fitness functions, lint/ typecheck e **revisão independente**; dispensam-se os artefatos de planejamento e a doc viva **exceto** se a mudança tocar uma jornada. Bug **exige um teste que o reproduz** antes do fix.

Bloco obrigatório para ações irreversíveis (raia infra, §3)

Além da DoD acima, toda ação irreversível (migração destrutiva, deploy, exclusão de dados) exige — materializando a "reversibilidade engenheirada":

- [] **backup/snapshot** antes de qualquer ação destrutiva;
- [] **dry-run + validação em staging** antes da produção;
- [] **estratégia de rollback** documentada no runbook (soft-delete quando aplicável);
- [] **aprovação humana explícita** (classe de risco "financeira/irreversível", §8).

8. Mapa de gates humanos inegociáveis

Reaproveita a **taxonomia de classes de risco** da Constituição (Princípio IV). O agente age sozinho em risco baixo; o humano **DEVE** aprovar a partir de "alteração".

CLASSE DE RISCO	EXEMPLO	AGENTE PODE SOZINHO?	GATE HUMANO
Leitura	Ler código, buscar, explorar	✔ Sim	Não
Leitura sensível	Dados com PII/segredos	⚠ Com política + mascaramento	Revisão
Criação reversível	Nova feature em branch, rascunho	✔ Sim	Merge
Alteração	Refactor amplo, mudança de contrato	✗ Não	Aprovação com resumo
Exclusão / ação externa	Apagar dados, chamar API externa, push	✗ Não	Confirmação forte
Financeira / irreversível	Deploy em produção, migração destrutiva	✗ Não	Dupla aprovação / reautenticação
Lote / cross-tenant / admin	Migração em massa, ação administrativa	✗ Bloqueado	Workflow humano formal

Gates humanos inegociáveis, sempre: **aprovar a spec, aprovar o plan (Constitution Check), aprovar o merge, autorizar deploy/migração**. Nenhum desses é delegável a agente.

9. Rastreabilidade e fluxo end-to-end

```
intenção → spec.md (NNN) → plan.md (+Constitution Check) → ux-design.md → tasks.md
→ implementação (Dev-agent) → testes + fitness functions (CI)
→ revisão independente (Review-agent) + segurança
→ docs vivas (journey/ADR/changelog) → PR → gate humano de merge → release
```

Na **raia leve** (§3) o caminho encurta para `intenção → code → teste → revisão independente → PR (o artefato) → merge`, sem `spec/plan/tasks`.

Cada feature mantém o elo **spec NNN ↔ PR ↔ testes ↔ journey** explícito (parte da DoD). Isso dá a rastreabilidade `decisão→código→verificação` que a governança leve exige, sem ferramenta pesada.

10. O que NÃO adotamos (anti-processo / YAGNI)

Explicitado para evitar processo especulativo (Constituição, Governance/YAGNI):

- **Scrum completo** (Scrum Master, sprint planning, daily de time) — cerimônia de papel para solo+IA. Ficamos com shaping + cooldown + retro.
- **Backlog formal** — Shape Up mostra que ideias importantes voltam via re-shaping; não acumulamos backlog.
- **Segunda ferramenta de spec-driven (ex.: OpenSpec)** — avaliada e descartada: não manter duas ferramentas SDD. O que valia era a ideia do modelo *delta* para mudanças pequenas, absorvida como **raia leve dentro do Spec Kit** (§3).
- **RFC process pesado** — o ADR já cobre decisão + contexto + consequências.
- **C4 formal e instrumentação de métricas DORA** — hoje mermaid + linguagem-ubíqua e o CI verde bastam; reavaliar quando o time/escala justificar (Fase 2).
- **Estimativas de prazo** — usamos apetite (tempo fixo, escopo variável).

11. Evolução deste documento

Este documento é **versionado** e evolui como a Constituição: mudanças materiais sobem versão (MINOR: novo papel/cerimônia/artefato/raia; PATCH: clarificação) e são registradas em ADR. A **retro de ciclo** é a fonte primária de emendas — toda regra nova nasce de um erro recorrente observado. Revisão obrigatória ao fim de cada fase do projeto.

Histórico: 1.0.0 (2026-07-22, ADR 0004) fundação · 1.1.0 (2026-07-22, ADR 0005) raias de trabalho (leve/plena/infra), spec de infra com gates de reversibilidade, OpenSpec avaliado e descartado · 1.2.0 (2026-07-22, ADR 0006) aplicação e enforcement (§12): PR template com a DoD, gate de CHANGELOG na CI, comando `/dod`.

12. Aplicação e enforcement (como garantir que o modelo é aplicado)

A garantia não vem de disciplina nem de memória — vem de **tornar cada critério executável e bloqueante** (o próprio [8] aplicado ao modelo). Cada item tem um mecanismo, dividido entre **mecânico** (hard gate, bloqueia sozinho) e **julgamento** (checklist + aprovação humana):

ITEM	MECANISMO	ONDE
Testes + fitness functions + build/typecheck	Hard gate (CI) — bloqueia o merge	<code>.github/workflows/ci.yml</code>
Entrada no CHANGELOG	Hard gate (CI) — job <code>changelog</code> (bypass: label <code>skip-changelog</code>)	<code>ci.yml</code> + <code>CHANGELOG.md</code>
Secret scanning	Setting nativo do GitHub (secret scanning + push protection)	configuração do repo
DoD/DoR, raia, rastreabilidade, gate de risco	Checklist obrigatório	<code>.github/pull_request_template.md</code>
Self-check do agente antes de "pronto"	Comando <code>/dod</code>	<code>.claude/commands/dod.md</code>
DoR (spec pronta) + Constitution Check	Templates spec-kit + Constituição	<code>.specify/templates/</code> , <code>/speckit.plan</code>
Ler Constituição/modelo antes de agir	Diretriz	<code>CLAUDE.md</code> / <code>AGENTS.md</code>

A divisão é a mesma do [5] : o que é mecânico (R/C automatizáveis) vira hard gate na máquina; o que exige julgamento — "é a coisa certa?", classe de risco ([9]) — fica no checklist do PR + aprovação humana (o A). *Pendências (follow-up)*: habilitar secret scanning nativo; avaliar lint (ruff/eslint).

PARTE III

Handbook

*Fundamentos por elemento — teoria, frameworks
avaliados e recomendação.*

Handbook do Modelo Operacional

— Fundamentos por elemento

Manual de referência dos elementos do modelo operacional (docs/governance/modelo-operacional.md). Um capítulo por elemento, cada um com a mesma anatomia: fundamentação teórica → frameworks avaliados → recomendação de uso no nosso contexto (1 humano orquestrando N agentes de IA).

Este handbook é **didático e fundamentado**; complementa — não substitui — os outros três documentos:

DOCUMENTO	PAPEL
governance/modelo-operacional.md	Normativo — a regra vigente (o quê é obrigatório)
handbook/ (aqui)	Fundamentos — por que a regra é essa; teoria + frameworks + recomendação
research/jornada-aprendizado-modelo-operacional.md	Diário — os insights construídos ao criticar cada elemento
research/resultado-pesquisa-**-avaliacao.md	Pesquisa — a síntese citada que embasa tudo

Anatomia obrigatória de cada capítulo (7 seções)

- Pergunta central** — a pergunta que o capítulo responde.
- Fundamentação teórica** — o conceito, sua origem e o princípio que o sustenta.
- Frameworks / abordagens avaliados** — o que existe lá fora, comparado, com veredito.
- Recomendação de utilização** — como aplicamos no contexto 1 humano + N agentes; ligação com o modelo-operacional.md e com a Constituição.
- Conexões** — como o elemento se liga aos demais capítulos.
- Insight da jornada e impacto no modelo** — o que refinamos ao avaliar criticamente e o efeito normativo (ADR/versão), com link ao diário.
- Fontes** — citadas, com data.

Índice (ordem de leitura = ordem de dependência)

CAP.	ELEM.	TÍTULO	RESUMO (IDEIA CENTRAL)
01	[1]	Princípio operacional central	IA escreve · humano decide/aprova · verificação independente valida. O que torna o irreversível seguro de delegar é a reversibilidade engenheirada .
02	[10]	Evidência: DORA/SPACE	Velocidade e estabilidade não são trade-off . Alavanca: lote pequeno + reversibilidade . Bússola, não painel.

CAP.	ELEM.	TÍTULO	RESUMO (IDEIA CENTRAL)
03	[2]	Spec-Driven	A spec é a fonte de verdade (input que gera código, não descrição). Raias leve/plena/infra por ambiguidade × raio × irreversibilidade .
04	[3]	Fluxo agentic e economia de contexto	explore→plan→code→commit + subagentes + /clear + revisor fresco = economia de contexto . A spec é o contexto integrador.
05	[4]	Orquestração de agentes	Map-reduce cognitivo : reduce do orquestrador; corte nas costuras (bounded contexts). Espectro workflow↔agent; menor autonomia que resolve .
06	[5]	Papéis e RACI	Papéis como modos. Delega-se R/C/I ; nunca o A . Humano Accountable pela política/gates/critérios — não por item .
07	[6]	Cerimônias e cadência	Cerimônia = função , não reunião. Retro amplificada por agentes. WIP = atenção humana . Shape Up + Kanban.
08	[7]	Entregáveis e artefatos	Artefato vive se é input consumido com forcing function (ou imutável, como o ADR). Não duplicar função já servida.
09	[8]	Definition of Ready / Done	"Done" precisa ser verificável autonomamente ; converter julgamento em check . Verde ≠ certo — global fica com o humano.
10	[9]	Gates e classes de risco	Gate proporcional a irreversibilidade × impacto . Uniforme = funil ou catástrofe. Reversibilidade rebaixa a classe .
11	[11]	Rastreabilidade	spec ↔ PR ↔ teste ↔ journey = memória durável (sobrevive ao reset do agente). Emerge do workflow, sem ferramenta.
12	[12]	Governança leve	Aprende sem inchar : núcleo firme + periferia evoluível + YAGNI . A jornada <i>foi</i> o loop de governança em ação.

Nota: o número do capítulo (ordem de leitura) ≠ o rótulo do elemento [x] (nossa linguagem compartilhada na jornada). A tabela mapeia os dois.

Formatos

- 📄 **PDF completo** (livro A4, capa + 12 capítulos): [maestro-handbook.pdf](#)
- 🎵 Apresentação **executiva** (HTML): [apresentacao-executiva-maestro.html](#)
- 📐 Caderno **técnico** (HTML): [apresentacao-tecnica-maestro.html](#)

Capítulo 01 — Princípio operacional central (elemento [1])

IA para explorar, propor e escrever; humano para especificar, decidir e aprovar; testes, gates e revisão independente para validar.

1. Pergunta central

Se um agente *é capaz* de tomar qualquer decisão — inclusive registrar alternativas e racional impecáveis — **o que ainda exige o humano?**

2. Fundamentação teórica

Quando é o agente que escreve o código, o eixo de controle se desloca: o que garante que o software faz o que o negócio quer não é mais digitar código — é a clareza da **intenção** e a força da **verificação**. Três pilares:

- 1. **A intenção vive na especificação**, não no código nem no prompt (base do [2]). O humano dirige e refina; o agente escreve.
- 2. **Quem executa não é quem verifica**. A verificação passa por um revisor independente, de preferência em *contexto fresco* — segregação de funções aplicada a agentes.
- 3. **Prove, não declare**. "Pronto" exige evidência que o agente gere e um gate confira.

Accountability x capacidade. Capacidade de *raciocinar* não é o mesmo que *responder pelas consequências*. O gate humano não mede a qualidade do raciocínio do agente; ele **localiza a responsabilidade**.

Ex-ante x ex-post. Um registro auditável é *ex-post* — descreve a decisão depois de tomada. Um gate é *ex-ante* — barra antes de executar. Para uma ação **irreversível**, auditar não impede o dano.

A alavanca real: reversibilidade. O que torna uma ação irreversível *segura de delegar* não é a aprovação — é convertê-la em reversível (backup, dry-run, staging, soft-delete). O gate humano sempre foi um **proxy** de "torne reversível ou olhe antes".

3. Frameworks / abordagens avaliados

ABORDAGEM	O QUE OFERECE	VEREDITO
Human-in-the-loop / mixed-initiative (Horvitz)	Humano no ponto de decisão para ações consequentes	Adotado — espinha do princípio
Política declarativa allow/deny/ask (Const. §IV; OpenAI Agents; OPA)	Decidir <i>classes</i> de ação autônoma, não cada instância	Adotado — humano decide a política, que fica registrada
ADR (Nygard)	Registro auditável: decisão + contexto + consequências + alternativas	Adotado — protocolo de registro de decisão

ABORDAGEM	O QUE OFERECE	VEREDITO
Reversibility patterns (backup/dry-run/staging/soft-delete; git checkpoint/rewind)	Tornar o irreversível reversível	Adotado — requisito de DoD para ação irreversível
Taxonomia de classes de risco (Const. §IV)	Grada o gate por risco	Adotado — mapa dos gates humanos

4. Recomendação de utilização (1 humano + N agentes)

- **O humano decide a política de delegação, e ela vai ao registro** — a responsabilidade sobe da instância para a política (`allow/deny/ask`).
- **Verificação independente valida** — revisor em contexto fresco ([3]).
- **Ação irreversível exige reversibilidade engenheirada** antes de delegar (DoD de infra — `modelo-operacional.md` §7, §8).
- Gates indelegáveis: aprovar spec, plan, merge, autorizar deploy/migração.

5. Conexões

- [2] **Spec-Driven** — a spec é onde a intenção humana vive.
- [3] **Fluxo agentic** — reversibilidade = o checkpoint/rewind/git; verificação = revisor fresco.
- [10] **DORA** — "recuperável > cauteloso" é a evidência empírica da reversibilidade.
- [9] **Gates/risco** — a taxonomia que operacionaliza o gate.

6. Insight da jornada e impacto no modelo

Refinado ao ser **criticado**: de "toda decisão pode ser automatizada com registro" → "o humano delega e a delegação vai ao registro" → **reversibilidade é o que torna o irreversível seguro de delegar**. Sem impacto normativo direto (o princípio já constava); alimentou os gates de reversibilidade formalizados no [2] /§7. Diário: [1] em `research/jornada-aprendizado-modelo-operacional.md`.

7. Fontes

- Anthropic — *Building effective agents*: <https://www.anthropic.com/engineering/building-effective-agents>
- Claude Code — *Best practices*: <https://code.claude.com/docs/en/best-practices>
- E. Horvitz — *Principles of Mixed-Initiative UI*: <https://www.microsoft.com/en-us/research/publication/principles-mixed-initiative-user-interfaces/>
- M. Nygard — *Documenting Architecture Decisions*: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- OWASP — *LLM01 Prompt Injection*: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
- Constituição da Plataforma (Princípio IV).

Capítulo 02 — Evidência:

DORA / SPACE (elemento [10])

A base empírica embaixo do [1] : **velocidade e estabilidade não são um trade-off.**

1. Pergunta central

A intuição comum diz "quanto mais rápido eu entrego, mais eu quebro". **Velocidade e estabilidade são mesmo opostas — ou andam juntas?**

2. Fundamentação teórica

O programa DORA (DevOps Research and Assessment) mediu milhares de equipes e definiu **quatro métricas**, em dois pares:

- **Throughput**: frequência de deploy · lead time (commit→produção)
- **Estabilidade**: change fail rate · tempo de recuperação (rollback) — hoje às vezes com uma 5ª, *rework rate*.

Achado central, contraintuitivo: os **mesmos** times de elite são rápidos **e** estáveis, simultaneamente. "O trade-off real, no longo prazo, é entre *software melhor mais rápido e software pior mais devagar*." As quatro métricas são **outcomes** dirigidos por **capacidades**: CI, trunk-based development, testes automatizados, revisão de código, deploys pequenos.

A alavanca única: tamanho de lote pequeno. Mudança pequena → deploys mais vezes (frequência ↑), lead time curto, fácil de testar/revisar (falha ↓) e, quando falha, fácil de reverter (recuperação ↑). Padronização + automação é o que torna o lote pequeno barato; **reversibilidade** ([1]) é o que torna a falha barata. Por isso as quatro se movem juntas em vez de brigar.

3. Frameworks / abordagens avaliados

FRAMEWORK	FOCO	VEREDITO PARA SOLO+IA
DORA (4 keys + capabilities)	Desempenho de entrega + capacidades causais	Adotado como bússola qualitativa — lead time e change fail rate são os dois que mais importam para um solo
SPACE	Produtividade multidimensional (Satisfação/Performance/Atividade/Comunicação/Eficiência)	Referência — lembra que "não há uma métrica única de produtividade"; pouca prescrição
DX Core 4	4 métricas prescritivas (Speed/Effectiveness/Quality/Impact)	Observado — cuidado com métricas gaméaveis (ex.: "PRs por dev")

FRAMEWORK	FOCO	VEREDITO PARA SOLO+IA
Painel instrumentado de métricas	Dashboards de DORA	YAGNI agora — sem a quem comparar; instrumentar é débito prematuro

4. Recomendação de utilização (1 humano + N agentes)

- Usar DORA como **bússola de saúde do fluxo**, não como painel de RH ("consigo levar uma spec a produção rápido e sem quebrar?").
- Focar **lead time** e **change fail rate**; recuperação vem da reversibilidade ([1]).
- **Não instrumentar métricas** ainda (YAGNI). As capacidades (CI verde, trunk-based, testes, diffs pequenos, revisão independente) já são princípio da Constituição (V) e da DoD — a evidência só confirma que são as certas.

5. Conexões

- [1] — "recuperável > cauteloso" é a leitura empírica da reversibilidade.
- [3] — rollback rápido = o checkpoint/rewind do fluxo agentic.
- [8] **DoR/DoD** — as capacidades DORA são exatamente os itens da DoD.

6. Insight da jornada e impacto no modelo

Analogia do aprendiz: **linha de produção cognitiva** — como a fábrica pré-Ford era lenta até padronizar componentes, aqui a padronização de processo/entregáveis reduz fricção e variação. Complemento: a linha do Ford *não tinha undo barato*; a cognitiva tem — **lote pequeno + reversibilidade = rápido e estável**. Sem impacto normativo (confirma o modelo); justifica por que a DoD e as raias não são burocracia. Diário: [10] .

7. Fontes

- DORA — *DORA metrics (four keys)*: <https://dora.dev/guides/dora-metrics-four-keys/>
- Swarmia — *Comparing DORA, SPACE and DX Core 4*: <https://www.swarmia.com/blog/comparing-developer-productivity-frameworks/>

Capítulo 03 — Desenvolvimento dirigido por especificação (elemento [21])

A keystone da mecânica: a spec é a fonte de verdade, não o código nem o prompt.

1. Pergunta central

No mundo tradicional, a fonte de verdade é o código — a spec e a doc *apodrecem*. O Spec-Driven inverte isso. **O que muda, quando é o agente que escreve, que faz a spec parar de apodrecer e passar a ser mais confiável que o código?**

2. Fundamentação teórica

Inversão de dependência. Em desenvolvimento tradicional a doc *descreve* o código (está a jusante → apodrece). No Spec-Driven a spec é o **input que gera** o código: se você quer código novo, precisa mudar a spec primeiro. A obsolescência deixa de ser opcional e passa a quebrar a geração — a spec vira "verdade executável".

Fluxo: `specify → clarify → plan → tasks → implement`. A spec descreve **o quê e por quê** (valor, critérios de aceite testáveis); o plan descreve **como**. Prompt vago "força o modelo a adivinhar milhares de requisitos não ditos".

Nem toda mudança merece uma spec. O valor de uma spec escala com **ambiguidade × raio de impacto × irreversibilidade**. Quando os três são baixos, a spec é cerimônia de papel (YAGNI). Daí as **raias** (`modelo-operacional.md` §3).

3. Frameworks / abordagens avaliados

FERRAMENTA	MODELO	VEREDITO
GitHub Spec Kit	1 spec/feature; fases com gate; cross-feature forte	Adotado — base do fluxo; casa com constituição + specs numeradas
OpenSpec (Fission-AI)	<i>Delta</i> (<code>Propose→Apply→Archive</code>); leve; sem Python	Descartado como 2ª ferramenta — a ideia do delta foi absorvida como raia leve
Kiro / Tessl / "specs as source of truth"	Variações de SDD assistido	Observados — mesmo princípio; sem adoção
Prompt ad-hoc	Sem spec	Rejeitado — produz código que compila e erra a intenção

4. Recomendação de utilização (1 humano + N agentes)

- **Uma ferramenta SDD só (Spec Kit).** A raia leve (modelo delta) mora dentro dela.
- **Três raias:** leve (bug/typo — o PR é o artefato, bug exige teste que reproduz), plena (feature/contrato — Spec Kit completo), infra (sempre plena + gates de reversibilidade).
- Regra de desempate: **na dúvida, é plena; infra/migração nunca são leves.**

5. Conexões

- [3] / [4] — a spec é o **contexto integrador** que sobrevive ao `/clear` e cruza toda fronteira de subagente (antídoto ao ótimo local).
- **DDD/hexagonal** — dá as fronteiras (bounded contexts) onde a spec corta o trabalho.
- [8] **DoR** — "spec pronta" é a Definition of Ready.

6. Insight da jornada e impacto no modelo

"Documentação atualizada" era o *sintoma*; a causa é a **inversão de dependência**. A crítica "bug dá pra simplificar" + a avaliação do OpenSpec **produziram mudança normativa real**: `modelo-operacional.md` **v1.1.0** (§3 raías; spec de infra com gates)

- **ADR 0005**. Diário: [2] .

7. Fontes

- GitHub Blog — *Spec-driven development with AI* (2025-09): <https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/>
- GitHub Spec Kit: <https://github.com/github/spec-kit>
- OpenSpec (Fission-AI): <https://github.com/Fission-AI/OpenSpec>
- Spec Kit vs OpenSpec: <https://intent-driven.dev/knowledge/spec-kit-vs-openspec/>

Capítulo 04 — Fluxo agentic e economia de contexto (elemento [3])

Como o trabalho roda: **explore** → **plan** → **code** → **commit**, subagentes, `/clear` e revisor em contexto fresco — quatro peças, uma restrição.

1. Pergunta central

O fluxo agentic tem quatro peças que *parecem* boas práticas soltas. **Qual restrição física única elas estão todas contornando?**

2. Fundamentação teórica

A janela de contexto do agente é **finita**, e a performance **degrada à medida que ela enche** — o agente "esquece" instruções antigas e erra mais. O inimigo não é a *falta* de contexto; é o **acúmulo** (becos sem saída, arquivos irrelevantes) que afoga o sinal.

As quatro peças são, todas, **economia de contexto**:

PEÇA	O QUE FAZ COM O CONTEXTO
explore→ plan →code	o <i>plan</i> comprime a exploração no essencial
subagentes	leem em janela separada; só volta o resumo — o principal fica magro
<code>/clear</code>	despeja ruído entre tarefas não relacionadas
revisor fresco	não contaminado pelos becos sem saída; vê só diff + critério

Limite (não exagerar): resetar demais esquece o básico; delegar demais leva o subagente ao **ótimo local** (otimiza a fatia, quebra o todo). Disciplina madura: **preserve o contexto que sustenta (load-bearing), descarte só o ruído**. O contexto integrador é a **spec/plan** ([2]).

3. Frameworks / abordagens avaliados

PADRÃO/PRÁTICA	O QUE OFERECE	VEREDITO
explore-plan-code-commit (Claude Code)	Separa pesquisa de execução → não resolve o problema errado	Adotado (raia plena)
Writer/Reviewer (contexto fresco)	Revisor sem viés de quem escreveu	Adotado — é a revisão independente da DoD
Subagentes	Investigação/verificação em janela isolada	Adotado — cortados por bounded context
Revisão adversarial do diff	Modelo fresco tenta refutar antes do "pronto"	Adotado — <code>/code-review</code> como gate
<code>/clear</code> / compactação	Reset de contexto entre tarefas	Adotado — higiene de contexto

4. Recomendação de utilização (1 humano + N agentes)

- **Plan comprime** a exploração antes de implementar (raia plena).
- **Subagentes por bounded context**, cada um recebendo a spec (contexto integrador).
- **Revisão independente em contexto fresco** é item obrigatório da DoD (todas as raias).
- Preservar o load-bearing (objetivo, invariantes, contratos); descartar o ruído.

5. Conexões

- [2] — mesma espinha: a spec é o que sobrevive ao reset e cruza fronteiras.
- [4] — o ótimo local abre a necessidade de orquestração (reduce + corte por costura).
- [1] — checkpoint/rewind é a reversibilidade aplicada ao contexto.

6. Insight da jornada e impacto no modelo

Do "falta de contexto" ao **acúmulo/foco**; das 4 peças a **uma ideia (economia de contexto)**; do "delegar" ao **ótimo local**. Confirmou decisões que já estavam no modelo (/clear entre tarefas, revisão em contexto fresco no §4) — o aprendiz codificou o *quê* antes de entender o *porquê*. Diário: [3] .

7. Fontes

- Claude Code — *Best practices* (economia de contexto, subagentes, revisão adversarial): <https://code.claude.com/docs/en/best-practices>
- Anthropic — *Building effective agents*: <https://www.anthropic.com/engineering/building-effective-agents>

Capítulo 05 — Padrões de orquestração de agentes (elemento [4])

Como coordenar subagentes sem cair em ótimo local: **map-reduce cognitivo** ao longo do espectro **workflow ↔ agent**.

1. Pergunta central

Cinco subagentes devolvem cinco fatias, cada um com a spec em mãos. Elas não se encaixam sozinhas. **O que precisa acontecer depois — e quem, estruturalmente, faz isso?**

2. Fundamentação teórica

A peça que falta depois do *map* (workers em paralelo) é o **reduce cognitivo**: reconciliar as fatias (contratos, coerência com a spec). Não é `cat fatia1 fatia2` — é *julgamento*, e só pode ser feito por quem tem o **contexto global**: o **orquestrador**. Padrão: **orchestrator-workers**.

O corte é mais traiçoeiro que o reduce. O map-reduce clássico assume splits independentes; em código quase nunca são (porta e adaptador dividem um contrato). Se você corta nas **costuras erradas**, nenhum reduce salva. As costuras certas são os **bounded contexts** (DDD) — cortar ao longo das fronteiras arquiteturais é o que torna a paralelização segura.

Espectro workflow ↔ agent. *Workflow* = você fixa o caminho (previsível, foco, sinergia). *Agent* = o LLM decide o corte na hora (flexível, mas pode errar o corte — ótimo local — e varia). Regra de ouro: **use a menor autonomia que resolve** ("comece simples; adicione autonomia só quando o simples comprovadamente falha").

3. Frameworks / abordagens avaliados

PADRÃO	O QUE É	TIPO	ONDE VIVE NO MODELO
Prompt chaining	passos fixos em sequência	workflow	fluxo Spec Kit <code>specify→...→implement</code>
Routing	classifica a entrada → handler certo	workflow	as raias (§3)
Parallelization	fatias independentes / N tentativas (voting)	workflow	subagentes por bounded context
Orchestrator-workers	corte dinâmico + reduce	agent	papel Orquestrador (§4)
Evaluator-optimizer	gera → crítica → refina	workflow c/ loop	Writer/Reviewer + <code>/code-review</code>
Autonomous	LLM aberto + guarda-corpos	agent	evitado (YAGNI; exige guardrails pesados)

4. Recomendação de utilização (1 humano + N agentes)

- **Default é workflow** (chaining/routing/parallel) — barato, previsível, depurável.
- **Parallelize por bounded context**; o **Orquestrador (humano)** faz o reduce.
- Suba para orchestrator-workers/autonomous **só** quando o corte genuinamente não pode ser predefinido.
- **Melhor arquitetura → mais tempo no lado barato do espectro.** Boas fronteiras compram previsibilidade.

5. Conexões

- [3] — resolve o *ótimo local* deixado em aberto pelo fluxo agentic.
- [2] / **DDD** — as fronteiras onde se corta; a spec como norte de cada worker.
- [1] — evaluator-optimizer é o revisor independente institucionalizado.

6. Insight da jornada e impacto no modelo

Analogia do aprendiz: "**map-reduce, mas cognitivo**"; e o insight de que **fixar o corte** reduz escopo e ganha sinergia (workflow > agent quando o corte é conhecido). Sem impacto normativo direto; mapeia os padrões sobre papéis/raias já existentes. Diário: [4].

7. Fontes

- Anthropic — *Building effective agents* (workflow×agent; 6 padrões; menor autonomia): <https://www.anthropic.com/engineering/building-effective-agents>
- Claude Code — *Best practices* (subagentes, fan-out, Writer/Reviewer): <https://code.claude.com/docs/en/best-practices>

Capítulo 06 — Papéis e responsabilidades (elemento [5])

Preservar os contrapesos de um time — **decide ≠ executa ≠ verifica** — sem ter um time: papéis como modos de trabalho numa **matriz humano × agentes**.

1. Pergunta central

Num time, decidir, executar e verificar são **pessoas diferentes** — é justamente essa separação que cria o contrapeso. Você é **uma** pessoa + N agentes. **Como preservar os contrapesos sem colapsar os três numa só cabeça?**

2. Fundamentação teórica

Papéis são modos de trabalho, não cargos. Cada papel é exercido por humano, agente, ou os dois com gate. Num time minúsculo os papéis **acumulam-se** — o risco é perder os contrapesos ao concentrar tudo numa pessoa.

RACI dá o vocabulário: **R**esponsible (executa), **A**ccountable (responde/aprova), **C**onsulted (verifica/consultado), **I**nformed. Regra clássica: **exatamente um Accountable por tarefa** (um pescoço a apertar).

Delegabilidade em solo+IA:

- **R (executa)** → agente (dev/spec/plan/etc.).
- **C (verifica)** → agente **independente**, em contexto fresco ([3]) — nunca o mesmo que executou, nunca só o próprio humano (senão "eu decidi, eu confiro" mata a independência).
- **I (informado)** → logs / ExecutionTrace.
- **A (Accountable)** → **humano, sempre**. Um agente raciocina, executa e revisa, mas **não responde pelas consequências** — accountability não delega ([1]).

A armadilha do funil. Se o humano é A e confere à mão cada saída de agente, ele recria "fazer tudo sozinho" — o A vira gargalo. Resolução: **o humano é Accountable pela política, pelos gates e pelos critérios — não por cada instância.** Ele projeta os trilhos (allow/deny/ask + DoD verificável + critérios da spec); os agentes operam dentro deles; ele faz spot-check e responde pelos resultados. É a *política de delegação* do [1] aplicada a papéis — o que faz o modelo escalar.

3. Frameworks / abordagens avaliados

ABORDAGEM	O QUE OFERECE	VEREDITO
Matriz RACI/RASCI	Quem executa/responde/consulta/informa por tarefa	Adotado — adaptado a humano × agentes
Regra "um Accountable"	Responsabilidade não se dilui	Adotado — o humano é o A único e fixo no risco

ABORDAGEM	O QUE OFERECE	VEREDITO
Papéis Scrum (PO/SM/Dev Team)	Papéis de time cross-funcional	Parcial — PO = Steward humano; Scrum Master e "dev team" são cerimônia de papel para solo (ver [6])
Time T-shaped / cross-funcional	Largura de skills + profundidade	Reinterpretado — o agente cobre a largura (muitas skills); o humano guarda a profundidade da decisão

4. Recomendação de utilização (1 humano + N agentes)

- Adotar a **matriz de papéis-modo** do `modelo-operacional.md` §4: Product Steward, Arquiteto/Tech Lead, Spec/Plan/UX/Dev/QA/Review/Security/Tech-Writer-agents, Orquestrador.
- **R/C/I → agentes; A → humano**, e o humano é Accountable **pela política/gates/critérios**, não por item.
- **C sempre independente do R** (contexto fresco).
- Indelegável ao agente: aprovar spec, plan, merge, autorizar deploy/migração.

5. Conexões

- [1] — accountability indelegável + política de delegação (o "de que" o humano é A).
- [3] — o revisor em contexto fresco **implementa** o C independente.
- [8] **DoR/DoD** — critérios verificáveis são o que **permite** delegar o C a um agente.
- [9] **Gates/risco** — a taxonomia de risco define onde o A **tem** que agir.

6. Insight da jornada e impacto no modelo

Insights do aprendiz: "**delega-se tudo menos o A**" (porque accountability responde pelas consequências) e "**o humano é Accountable pela política, gates e critérios — não por cada item**" (o que evita o funil e faz escalar). Sem impacto normativo novo — consolida e fundamenta a matriz RACI já presente no §4. Diário: [5] .

7. Fontes

- Anthropic — *Building effective agents* (revisão humana crucial em agentes de código): <https://www.anthropic.com/engineering/building-effective-agents>
- Matriz RACI — prática consolidada de atribuição de responsabilidades em gestão.
- `docs/governance/modelo-operacional.md` §4 (papéis + RACI); Capítulo 01 ([1]).

Capítulo 07 — Cerimônias e cadência (elemento [6])

Cadência **Shape Up + Kanban**, não Scrum: cerimônia é **função**, não reunião — e o WIP se mede em **atenção do humano**, não em capacidade de agentes.

1. Pergunta central

Quais cerimônias sobrevivem quando é **um humano + N agentes** — e, já que os agentes paralelizam, **qual é o limite real do WIP?**

2. Fundamentação teórica

Cerimônia é a função que entrega, não a reunião. Para decidir o que cortar, pergunte "essa **função** ainda existe no meu contexto?".

CERIMÔNIA	FUNÇÃO	EM SOLO+IA
Planning	comprometer escopo	vira shaping/spec ([2])
Daily	sincronizar/destravar entre pessoas	função some → cortada
Review	inspecionar incremento	vira checkpoint de ciclo
Refinement	preparar backlog	cortada (sem backlog — Shape Up)
Retro	melhorar o processo	amplificada (ver abaixo)

Retro amplificada. Num time humano, a melhoria da retro é um hábito mole que decai. Com agentes, ela vira **instrução versionada** ([CLAUDE.md/skill/constituição](#)): **durável e executável**, aplicada automaticamente na próxima. É a cerimônia de **maior ROI** — cada erro recorrente de agente vira regra.

Cadência = Shape Up + Kanban. *Apetite* (tempo fixo, escopo variável — o oposto de estimar prazo; casa com YAGNI e diffs pequenos), *cooldown* (dívida/manutenção/curadoria), sem *backlog* formal (ideias importantes voltam via re-shaping), fluxo contínuo com **WIP limitado**.

O limite real do WIP é a atenção do humano. Não é a capacidade dos agentes nem a dependência entre tasks (essa limita o paralelo **dentro** de uma feature). Mesmo com 5 features de dependência zero, cada uma funila pelos **gates de decisão/aprovação/verificação do humano** ([5]). Portanto:

- **paralelize DENTRO** de uma feature já decidida (subagentes por bounded context);
- **não paralelize ACROSS** features ambíguas (multiplicam os gates);
- para escalar, **barateie os gates** (DoD verificável [8], reversibilidade [1]) — não abra mais frentes.

3. Frameworks / abordagens avaliados

FRAMEWORK	PAPÉIS/EVENTOS	VEREDITO PARA SOLO+IA
Scrum	PO/SM/Dev; planning, daily, review, retro, refinement	Parcial — mantém-se só a <i>função</i> de planning (→ spec) e retro; SM e daily são cerimônia de papel
Kanban	sem papéis/eventos; visualizar fluxo + limitar WIP	Adotado — WIP limitado (pela atenção humana) e fluxo contínuo
Shape Up	shapers/builders; appetite, ciclo 6sem + cooldown, betting, sem backlog	Adotado (esqueleto) — appetite + cooldown + sem backlog
Scrumban	híbrido	Observado — não precisamos do overhead do Scrum que ele carrega

4. Recomendação de utilização (1 humano + N agentes)

- **Cerimônias mínimas:** shaping/spec (planning real), execução por appetite, checkpoint de ciclo, cooldown, **retro** (→ regra versionada).
- **Cortar** daily e sprint planning (função de sincronização entre pessoas inexistente).
- **WIP baixo**, medido em **fluxos de decisão que o humano segura com qualidade**; paralelismo vai para **dentro** da feature.
- **Appetite** (tempo fixo, escopo variável): quando acaba, corta-se escopo — não se estende prazo.

5. Conexões

- [5] — o gargalo é o humano Accountable; o WIP é atenção humana.
- [4] — paralelizar *dentro* da feature (subagentes por bounded context).
- [2] — appetite = escopo variável = a raia certa por mudança.
- [8] / [1] — baratear os gates (DoD verificável, reversibilidade) é como se escala.

6. Insight da jornada e impacto no modelo

Insights do aprendiz: **cerimônia = função** (não reunião); **retro é a que fica mais valiosa com agentes**; dependência de tasks limita o paralelo **dentro** da feature, mas **o humano (atenção/decisão) é o gargalo do WIP através** de features. Sem impacto normativo novo — fundamenta a cadência do `modelo-operacional.md` §5. Diário: [6].

7. Fontes

- Basecamp — *Shape Up* (appetite, ciclo, cooldown, betting): <https://basecamp.com/shapeup>
- DORA — *four keys* (lote pequeno; velocidade x estabilidade): <https://dora.dev/guides/dora-metrics-four-keys/>
- `docs/governance/modelo-operacional.md` §5; Capítulos 01 ([1]), 06 ([5]).

Capítulo 08 — Entregáveis e artefatos (elemento [7])

Um artefato custa **duas vezes** (escrever + manter). Só vale o custo se for um **input consumido com forcing function** — ou imutável.

1. Pergunta central

A lista clássica é enorme (PRD, spec, plan, ADR, RFC, design doc, journey, runbook, changelog, C4...). **O que faz um artefato valer seu custo duplo — e o que o distingue de "cerimônia de papel" que apodrece?**

2. Fundamentação teórica

Todo artefato custa para **escrever** e para **manter atualizado**. O que "dá contexto e direção" é necessário, mas **não** distingue — um design doc também dá direção e apodrece. O que distingue é **mecânico**:

Um artefato sobrevive quando é um INPUT que algo downstream consome, com uma forcing function que falha barulhento quando ele está velho — ou quando é **imutável** (registra um ponto no tempo e nunca muda, como o ADR).

- A **spec** vive porque o agente **gera código dela**: velha → código errado → quebra visível ([2]).
- O **teste** vive porque a **CI** o consome: velho → vermelho.
- O **ADR** vive por **imutabilidade**: nunca precisa de update.
- O **journey** é **vivo por gate**: a Constituição (Princípio VII) regenera as capturas do build real e revisa a heurística **no mesmo PR** — sem esse gate, apodrece. É o caso canônico de *living documentation / docs-as-code*: doc desconectado do fluxo não tem forcing function.

Segunda metade da regra: não duplique função já servida por um artefato vivo. Um **PRD** avulso duplica a função da **spec**; um **design doc/RFC** avulso duplica **plan + ADR**. Sem consumidor próprio, apodrecem → cerimônia de papel (YAGNI).

3. Frameworks / abordagens avaliados

ARTEFATO	CONSUMIDOR	FORCING FUNCTION	VEREDITO
Spec (<code>spec.md</code>)	agente (gera código)	código errado se velha	Essencial
Plan (<code>plan.md</code>)	dev-agent / Constitution Check	divergência da impl	Essencial
Tasks (<code>tasks.md</code>)	dev-agent	efêmero (descarta pós-uso)	Essencial (efêmero)
Código + testes	CI	vermelho	Essencial
ADR	decisões futuras / onboarding	imutável	Essencial

ARTEFATO	CONSUMIDOR	FORCING FUNCTION	VEREDITO
Journey doc	gate de PR (capturas + heurística)	PR falha se não regenerar (P. VII)	Essencial (por gate)
Runbook	incidente / deploy	uso real / drills (fraco)	Essencial (disciplina)
Changelog	release	PR de release	Essencial (se gated)
DoR/DoD	agente / Stop-hook	gate de merge	Essencial
PR	revisão / merge	—	Essencial (na raia leve, é o artefato)
PRD avulso	(duplica a spec)	nenhuma	YAGNI
Design doc / RFC avulso	(duplica plan + ADR)	nenhuma	YAGNI
C4 diagrams	humano (leitura)	nenhuma	Fase 2 / YAGNI
Painel de métricas DORA	—	—	YAGNI

4. Recomendação de utilização (1 humano + N agentes)

- **Manter** apenas artefatos com consumidor + forcing function (ou imutáveis).
- **Não criar** artefato cuja função já é servida por um vivo (PRD→spec; design doc→plan+ADR).
- **Journey exige o gate** do Princípio VII para não apodrecer (não é "doc solto").
- Formalizar o **changelog** com forcing function no PR de release (pendência do modelo).

5. Conexões

- [2] — a spec é o arquétipo do "input consumido que não apodrece".
- [8] **DoR/DoD** — são artefatos-checklist verificáveis (habilitam o gate barato).
- [11] — a rastreabilidade liga os artefatos (spec↔PR↔testes↔journey).
- [12] — ADR é o artefato imutável de governança.

6. Insight da jornada e impacto no modelo

O que faz um artefato valer não é o **conteúdo** ("dá contexto") e sim o **mecanismo**: **consumidor + forcing function**, ou **imutabilidade** (ADR). E **não duplicar função já servida** (PRD/design doc avulsos = YAGNI). Fundamenta o catálogo do `modelo-operacional.md`

§6. Diário: [7].

7. Fontes

- M. Nygard — *Documenting Architecture Decisions* (ADR): <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- Simon Brown — *C4 model*: <https://c4model.com/>
- L. Mezzalana — *Documenting software architecture* (docs-as-code / living documentation): <https://lucamezzalana.medium.com/how-to-document-software-architecture-techniques-and->

best-practices-2556b1915850

- `docs/governance/modelo-operacional.md` §6; Constituição (Princípio VII, jornadas vivas).

Capítulo 09 — Definition of Ready / Done (elemento [8])

A DoD é o gate. Um gate barato é **verificável autonomamente** — mas verde ≠ certo.

1. Pergunta central

Baratear os gates é como se escapa do gargalo humano ([6]). **Que propriedade um critério de "done" precisa ter para um agente satisfazê-lo e um hook conferi-lo sem depender de você — e o que ele, mesmo verde, ainda não garante?**

2. Fundamentação teórica

A propriedade: verificável autonomamente. Um critério de "done" precisa produzir um **pass/fail objetivo** que o agente gera e um hook confere — "prove, não declare" ([11]) virado em máquina. *"Código limpo"* não é DoD (subjetivo); *"testes verdes + lint + revisão sem lacunas de correção"* é.

O trabalho é converter julgamento em check (harness design — ADR 0001):

- "código limpo" → lint + fitness functions + limite de complexidade;
- "boa UX" → tokens do design system + checklist heurístico;
- "seguro" → secret scanning + revisão de injeção/autorização.

DoR é a mesma ideia a montante: "pronto para começar" = a spec/plan é completa o bastante para o agente executar **sem adivinhar** (critérios testáveis, ambiguidades resolvidas, Constitution Check, apetite definido).

Necessária, não suficiente. Uma DoD verde garante a **peça local** — não garante:

- **(a) coerência global:** a jornada/todo ainda funciona e não regrediu (o **ótimo local** do [4] no nível da qualidade);
- **(b) a coisa certa:** verde não conserta requisito/spec errado. O modelo trata (a) puxando o global para dentro da DoD (journey, e2e, rastreabilidade [11]) + checkpoint humano; (b) fica com o **humano (o A do [5])**.

3. Frameworks / abordagens avaliados

ABORDAGEM	O QUE OFERECE	VEREDITO
DoR / DoD (Scrum/Kanban)	Portões de entrada/saída	Adotado — como checklists verificáveis (§7 do modelo)
Testing trophy (mais integração) vs test pyramid (mais unitário)	Onde investir teste	Trophy-leaning — integração por rota + contrato por porta pegam o "todo" melhor que só unitário
TDD / BDD	Teste antes; critério em linguagem de negócio	Adotado — bug exige teste que reproduz; caso de uso tem feliz + falha

ABORDAGEM	O QUE OFERECE	VEREDITO
Quality gates no CI + DevSecOps leve	Bloqueio automático	Adotado — fitness functions, lint/typecheck, secret scanning
Cobertura como meta numérica	% de linhas	Rejeitado — gamável; usamos "feliz + falha por caso de uso"

4. Recomendação de utilização (1 humano + N agentes)

- **DoR e DoD como checklists verificáveis** (`modelo-operacional.md` §7), fecháveis por Stop-hook/ `/goal` .
- **DoD reduzida na raia leve; bloco de reversibilidade** na raia infra.
- **Puxar o global para a DoD**: journey atualizada + e2e + rastreabilidade spec↔journey.
- **O humano guarda o irreduzível**: coerência global no checkpoint e "é a coisa certa".

5. Conexões

- `[1]` — "prove, não declare" é a raiz da DoD verificável.
- `[5]` — o humano é Accountable pelos **critérios**; a DoD é onde eles viram check.
- `[6]` — DoD verificável = gate barato = fuga do gargalo humano.
- `[4]` / `[11]` — verde local ≠ todo correto; rastreabilidade/journey puxam o global.

6. Insight da jornada e impacto no modelo

Insights do aprendiz: "**verificável autonomamente**" e "**a peça é local — ainda é preciso ver se atendeu o requisito da jornada / não comprometeu o conjunto maior**" (o ótimo local reaparecendo). Fundamenta a DoR/DoD do `modelo-operacional.md` §7. Diário: `[8]` .

7. Fontes

- Claude Code — *Best practices* (dê ao agente um check que ele rode; verificação como gate): <https://code.claude.com/docs/en/best-practices>
- DORA — *four keys* (testes/CI como capacidades): <https://dora.dev/guides/dora-metrics-four-keys/>
- Kent C. Dodds — *The Testing Trophy* (conceito; ênfase em integração).
- `docs/governance/modelo-operacional.md` §7; ADR 0001 (harness design).

Capítulo 10 — Gates humanos e classes de risco (elemento [9])

Um gate **uniforme** está sempre errado: pesado demais vira funil, leve demais deixa o irreversível escapar. O peso do gate escala com **irreversibilidade** × **impacto**.

1. Pergunta central

Você barateou os gates mecânicos ([8]) e guardou o humano para o global e o "é a coisa certa". Mas aprovar um typo ≠ aprovar uma migração destrutiva. **Por que graduar o gate por risco em vez de um gate uniforme — e qual eixo decide o peso?**

2. Fundamentação teórica

Os dois fracassos do gate uniforme:

- **pesado em tudo** → você aprova até o trivial → vira o **funil** (o gargalo do [6]);
- **leve em tudo** → uma ação irreversível **escapa** sem aprovação → a migração do [1] .

O eixo: **irreversibilidade** × **impacto (blast radius)**. O gate deve ser **proporcional ao risco**: automatize o baixo, escale o alto. Taxonomia oficial (Constituição §IV; modelo §8):

CLASSE	EXEMPLO	AGENTE SOZINHO?	GATE
Leitura	explorar, buscar	✓	nenhum
Leitura sensível	PII/segredos	⚠ política + máscara	revisão
Criação reversível	feature em branch	✓	merge
Alteração	refactor amplo, contrato	✗	aprovação com resumo
Exclusão / externa	apagar dados, push, API externa	✗	confirmação forte
Financeira / irreversível	deploy, migração destrutiva	✗	dupla aprovação
Lote / cross-tenant / admin	migração em massa	✗ bloqueado	workflow humano formal

Dois amarres:

1. **Mesmo eixo das raias ([2])**. **ambiguidade** × **raio** × **irreversibilidade** decide quanto **processo** (spec); **irreversibilidade** × **impacto** decide quanto **gate**. Mesma física: *processo* para construir, *gate* para agir.
2. **Reversibilidade rebaixa a classe**. Tornar a ação reversível (backup/staging/ soft-delete — [1]) move-a escada de risco **abaixo** → gate mais leve → menos funil.

3. Frameworks / abordagens avaliados

ABORDAGEM	O QUE OFERECE	VEREDITO
Política allow/deny/ask (OpenAI Agents approvals; OPA)	Decisão declarativa por classe	Adotado — a máquina de estados de ação (Const. §IV)
Taxonomia de classes de risco (Const. §IV)	Grada o gate	Adotado — base do [9]
Human approval for high-stakes (Anthropic)	Humano no ponto consequente	Adotado — gates indelegáveis
Least privilege / RBAC-ABAC	Autorização fora do LLM	Adotado — política decide, não o modelo
Gate uniforme (um approve pra tudo)	Simplicidade aparente	Rejeitado — funil ou catástrofe

4. Recomendação de utilização (1 humano + N agentes)

- **Gate proporcional** pela taxonomia do modelo §8; automatizar baixo risco, escalar alto, **bloquear** lote/cross-tenant/admin.
- **Indelegáveis** (sempre humano): aprovar spec, plan, merge, autorizar deploy/migração.
- **Autorização fora do LLM** (RBAC/ABAC/política) — o modelo nunca decide permissão.
- **Investir em reversibilidade** para rebaixar classes e reduzir gates pesados.

5. Conexões

- [1] — reversibilidade é o eixo e a alavanca (rebaixa a classe).
- [2] — mesma física das raías, aplicada a ações.
- [5] — o A humano age exatamente nos gates altos; delega os baixos.
- [6] / [8] — gate barato/automático no baixo risco evita o funil.

6. Insight da jornada e impacto no modelo

Insights do aprendiz: o eixo é **irreversibilidade x impacto**; um gate uniforme é funil ou catástrofe; logo **gate proporcional**. Fundamenta o mapa de gates do `modelo-operacional.md` §8 e a taxonomia da Constituição §IV. Diário: [9] .

7. Fontes

- Anthropic — *Building effective agents* (revisão humana para alto risco): <https://www.anthropic.com/engineering/building-effective-agents>
- OWASP — *LLM01 Prompt Injection* (dados também são hostis; política fora do LLM): <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
- Open Policy Agent: <https://www.openpolicyagent.org/docs>
- Constituição (Princípio IV, classes de risco); `docs/governance/modelo-operacional.md` §8.

Capítulo 11 — Rastreabilidade

spec ↔ PR ↔ testes ↔ journey (elemento [11])

Não é burocracia: é a **memória de longo prazo** do projeto — o porquê e o quê, sem re-derivar — e **emerge** do workflow, sem ferramenta pesada.

1. Pergunta central

Daqui a 3 meses um teste quebra e o **agente tem memória zero**. **O que a rastreabilidade te dá nesse momento — e como tê-la sem uma matriz de compliance que ninguém lê?**

2. Fundamentação teórica

Rastreabilidade devolve **o quê** (o que foi construído, o que o verifica) e **o porquê** (a decisão e o racional) **sem re-derivar**.

Por que é load-bearing em solo+IA: o agente **reseta** o contexto ([3]). Os artefatos ligados são a **única memória durável** — a rastreabilidade é o que reconstrói intenção + verificação depois do reset.

Dois sentidos:

- **para frente:** spec → foi construída? está verificada? (cobertura de intenção);
- **para trás:** sintoma → qual código → qual spec → por quê — que é **recuperação rápida** (a métrica de estabilidade do [10]).

Emergente, não um sistema. Você não constrói uma ferramenta de rastreabilidade; o elo **emerge** de cada artefato referenciar o próximo — spec **NNN** → PR cita a spec → teste nomeia o requisito → journey no mesmo PR → ADR para a decisão. A **DoD** ([8]) **força o elo**. É subproduto do workflow, não atividade separada.

3. Frameworks / abordagens avaliados

ABORDAGEM	O QUE OFERECE	VEREDITO
Matriz de rastreabilidade de requisitos	Rastreio formal req↔teste	Rejeitado — ferramenta pesada; matriz que ninguém mantém
Linking nativo issue↔commit↔PR (GitHub)	Elo barato no fluxo	Adotado — spec NNN e PR se referenciam
ADR ligado a componentes/C4	Decisão → sistema	Parcial — ADR sim; C4 é Fase 2
docs-as-code (cross-refs)	Links versionados	Adotado — journey/ADR/spec se citam

4. Recomendação de utilização (1 humano + N agentes)

- **Elo obrigatório na DoD:** spec **NNN** ↔ PR ↔ testes ↔ journey explícito.

- **Convenção de referência** (a spec numera; o PR cita; o teste nomeia o requisito; o ADR registra a decisão) — sem matriz, sem ferramenta.
- Otimizar o **sentido para trás** (sintoma → causa) — é o que acelera a recuperação.

5. Conexões

- [3] — os artefatos ligados são a memória que sobrevive ao reset de contexto.
- [8] — a DoD carrega e força o elo de rastreabilidade.
- [10] — o sentido "para trás" é a recuperação rápida (estabilidade).
- [7] / [12] — artefatos consumidos + ADR imutável = a trilha de decisões.

6. Insight da jornada e impacto no modelo

Insight do aprendiz: a rastreabilidade dá "**o porquê e o quê**" sem re-derivar; em solo+IA os artefatos ligados **são a memória** do projeto. Fundamenta o §8 (rastreabilidade) e a DoD (§7) do `modelo-operacional.md`. Diário: [11].

7. Fontes

- DORA — *four keys* (recuperação = trilha do sintoma à causa): <https://dora.dev/guides/dora-metrics-four-keys/>
- L. Mezzalira — *Documenting software architecture* (docs-as-code, cross-refs): <https://lucamezzalira.medium.com/how-to-document-software-architecture-techniques-and-best-practices-2556b1915850>
- `docs/governance/modelo-operacional.md` §8–§9.

Capítulo 12 — Governança leve (elemento [12])

Como o próprio sistema evolui e se mantém coerente — **aprendendo sem inchar**. A camada meta que fecha a jornada.

1. Pergunta central

Você tem um sistema de regras que **cresce** (constituição, ADRs, modelo versionado). **Qual é o modo de falha de um sistema de regras que cresce — e que força o combate?**

2. Fundamentação teórica

Modo de falha: bloat / ossificação. Regras acumulam, ninguém poda, e o conjunto vira o **processo pesado que a gente cortou** lá no começo.

A força contrária: YAGNI + poda + flexibilidade. A governança precisa de duas forças opostas equilibradas:

- **aprender** (adicionar regra) — via retro → ADR → nova versão;
- **ficar lean** (remover/não adicionar) — via YAGNI e poda do que não paga.

A Constituição já carrega essa poda: *"complexidade além do necessário DEVE ser justificada por escrito ou removida (YAGNI)."*

A arquitetura que dá flexibilidade: camadas com velocidades diferentes.

- **Núcleo firme** — a **Constituição** (princípios) muda devagar, por emenda versionada com Sync Impact Report;
- **Periferia evoluível** — **modelo operacional + handbook**, com **versão própria**, mudam rápido (por isso ancorados como *extensão subordinada* — Constituição v1.4.0);
- **Memória append-only** — **ADRs** imutáveis ([7]): o log de decisões que não apodrece;
- **Loop de aprendizado** — retro → regra versionada ([6]).

Governança aplica os próprios princípios a si mesma: é versionada, emendada sob gate humano ([9]), registrada em ADR ([7]) e podada por YAGNI. E os ADRs, sendo imutáveis mas **substituíveis** (um novo ADR pode superar outro), dão à decisão a mesma **reversibilidade** do [1]: nada fica preso, tudo é rastreável.

3. Frameworks / abordagens avaliados

ABORDAGEM	O QUE OFERECE	VEREDITO
Constituição versionada (spec-kit)	Fonte de verdade + emenda disciplinada	Adotado — núcleo firme
ADR (Nygard)	Log imutável de decisões + racional	Adotado — memória de governança
Working agreements / handbook	Convenções evoluíveis	Adotado — periferia rápida (este handbook)
RFC process pesado	Revisão formal ampla	Rejeitado — ADR cobre; YAGNI

ABORDAGEM	O QUE OFERECE	VEREDITO
Governança "adicionar sempre"	Mais regras = mais controle	Rejeitado — leva a bloat/ossificação

4. Recomendação de utilização (1 humano + N agentes)

- **Constituição firme em princípios**; modelo/handbook **evoluíveis e subordinados**.
- **ADR para toda decisão** que muda o sistema (imutável; superável por outro ADR).
- **Emenda via retro** (erro recorrente → regra); **poda via YAGNI** (regra que não paga sai).
- **Revisão obrigatória ao fim de cada fase** do projeto.

5. Conexões

- É a **meta-camada** de todos os elementos.
- [6] — a retro é o motor de emenda; [7] — ADR é a memória imutável; [9] — emenda é human-gated; [1] — ADR superável = reversibilidade da decisão.

6. Insight da jornada e impacto no modelo

Insights do aprendizado: **bloat/ossificação** é o risco; **YAGNI + poda + flexibilidade** é a força; governança precisa ser flexível para evoluir. **Meta**: esta própria jornada de aprendizado foi o [12] em ação — crítica → refino do modelo → ADR 0005 → versão v1.1.0 → ancoragem na Constituição v1.4.0. Diário: [12] .

7. Fontes

- M. Nygard — *Documenting Architecture Decisions*: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- GitHub Spec Kit (constituição versionada): <https://github.com/github/spec-kit>
- `.specify/memory/constitution.md` (Governance); `docs/adr/`; `docs/governance/modelo-operacional.md` §10-§11.

PARTE APÊNDICE

Decisões (ADRs)

O registro imutável das decisões de metodologia.

ADRs — Maestro

Architecture/Methodology Decision Records: decisões **imutáveis** (podem ser superadas por um ADR posterior, nunca editadas no mérito). Formato: contexto → decisão → consequências → fontes.

ADR	TÍTULO	STATUS
0004	Modelo operacional (papéis, cerimônias, artefatos)	Aceito
0005	Raias de trabalho e specs de infra	Aceito
0006	Enforcement da DoD e forcing function do CHANGELOG	Aceito
0007	Separação do Maestro em repositório próprio	Aceito

Nota de numeração

A sequência começa em **0004** porque os ADRs 0001-0003 do repositório de origem (`ghdaru`) eram específicos daquela plataforma (curadoria de skills, hospedagem, integração) e **não** pertencem à metodologia — não foram migrados. Os números 0004-0006 foram **preservados** para não quebrar as referências cruzadas no modelo e no handbook.

ADR 0004 — Modelo operacional (papéis, cerimônias, entregáveis e artefatos)

- **Status:** Aceito
- **Data:** 2026-07-22
- **Relacionado:** Constituição v1.3.0 (Princípios I, IV, V, VII e Governance);
`docs/governance/modelo-operacional.md` ; `docs/research/resultado-pesquisa-praticas-desenvolvimento-avaliacao.md`

Contexto

A Constituição define os **princípios** e o Spec Kit define o **fluxo técnico de uma feature**. Faltava a camada intermediária de governança: **quem faz o quê** (papéis/responsabilidades), **quando** (cerimônias/cadência) e **o que** se produz (entregáveis/artefatos com portões de qualidade) — num contexto singular: **um desenvolvedor humano orquestrando múltiplos agentes de IA**, não um time multi-pessoa.

Pesquisa profunda (2026-07-22, avaliada em `docs/research/resultado-pesquisa-praticas-desenvolvimento-avaliacao.md`) triangulou fontes primárias — GitHub Spec-Driven Development, Claude Code best practices, Anthropic "building effective agents", DORA, Shape Up, Swarmia — e concluiu:

- **Spec-Driven é o consenso de ponta:** quando o agente escreve o código, a especificação (não o prompt nem o código) é a fonte de verdade; o humano dirige e refina.
- **Time pequeno não precisa de Scrum:** Kanban/Shape Up (apetite fixo, escopo variável, sem backlog nem daily) preservam o valor sem a cerimônia.
- **Velocidade e estabilidade não são trade-off** (DORA); as 4 métricas são resultado de capacidades (CI, trunk-based, testes, revisão) que já são princípio nosso.
- **O gate humano + revisão independente é o contrapeso inegociável** de qualquer fluxo agentic sério.

Decisão

Adotar o **Modelo Operacional** documentado em `docs/governance/modelo-operacional.md`, que:

1. Define os **papéis como modos de trabalho** exercidos por humano, agente ou os dois com gate — preservando os contrapesos (quem *decide* ≠ *executa* ≠ *verifica*) ao mover o *verifica* para um **agente revisor em contexto fresco** e manter o *decide* sempre no **humano (Accountable fixo em toda linha de risco)**. RACI por etapa.
2. Estabelece a **cadência Shape Up + Kanban**: shaping/spec (planning real), execução por apetite fixo com WIP=1, checkpoint de ciclo, cooldown e **retro individual** — sem daily nem sprint planning. A retro alimenta emendas de processo (erro recorrente → regra versionada).
3. Cataloga os **entregáveis/artefatos** com dono, gate e localização, e formaliza **Definition of Ready** e **Definition of Done** como checklists verificáveis por agente (fecháveis por Stop-hook/ `/goal`).
4. Publica o **mapa de gates humanos inegociáveis** reaproveitando a taxonomia de classes de risco do Princípio IV (aprovar spec, plan, merge, deploy/migração são indelegáveis).

5. Torna explícita a **rastreabilidade** spec NNN ↔ PR ↔ testes ↔ journey como parte da DoD.

Fica **descartado por ora (YAGNI)**: Scrum completo, backlog formal, RFC pesado, C4 formal e instrumentação de métricas DORA — observados, reavaliáveis na Fase 2.

Consequências

- Existe uma referência única de "como o trabalho anda" acima da Constituição e do Spec Kit, sem duplicá-los (a Constituição prevalece em conflito).
- Definition of Ready/Done passam a ser checklists objetivos — habilitam automação de gate (Stop-hook, `/code-review` como gate de merge).
- O modelo é **versionado** e evolui via ADR + retro de ciclo; a próxima emenda provável é integrar a DoD a um hook de CI e materializar `CHANGELOG.md`.
- Candidatos de Fase 2 (C4, métricas DORA, DoD como hook) ficam registrados para não serem reinventados.

Fontes

Ver `docs/research/resultado-pesquisa-praticas-desenvolvimento-avaliacao.md` e `docs/research/fontes-consultadas.md` (seção 2026-07-22).

ADR 0005 — Raias de trabalho (leve/plena/infra) e specs de infra com gates de reversibilidade

- **Status:** Aceito
- **Data:** 2026-07-22
- **Relacionado:** docs/governance/modelo-operacional.md (v1.1.0); ADR 0004; Constituição v1.3.0 (Princípios I, IV, V)

Contexto

O modelo operacional v1.0.0 tratava todo trabalho como fluxo Spec Kit completo (`specify → clarify → plan → tasks → implement`). Na prática isso é caro demais para mudanças pequenas (bug, typo, rename, ajuste de log) — o fluxo completo vira cerimônia de papel (YAGNI). Ao mesmo tempo, infra e migrações são o oposto: alto raio de impacto e **irreversibilidade**, exigindo mais rigor, não menos.

Durante a avaliação surgiu o **OpenSpec** (Fission-AI) como alternativa leve ao Spec Kit, com modelo *delta* (`Propose → Apply → Archive` , mudanças como delta contra o comportamento atual). Decisão do mantenedor: **não manter duas ferramentas de spec-driven** — o custo operacional de duas SDDs não se justifica.

Princípio-guia derivado da jornada de aprendizado: **o valor de uma spec escala com ambiguidade × raio de impacto × irreversibilidade**; e o que torna uma ação irreversível segura de delegar não é a aprovação em si, mas a **reversibilidade engenheirada** (backup, dry-run, staging, soft-delete).

Decisão

1. Três raias de trabalho no modelo operacional (§3):

- **Leve (delta):** bug/typo/rename/log — sem `spec/plan/tasks` ; o **PR é o artefato**; bug exige um teste que o reproduz. DoD reduzida (mantém testes, fitness functions e revisão independente).
- **Plena (spec):** feature ambígua/contrato/cross-feature — Spec Kit completo.
- **Infra:** infra/migração/deploy — **sempre plena**, nunca leve, com gates de reversibilidade adicionais. Regra de desempate: na dúvida, é plena; infra/migração nunca são leves.

2. **Spec de infra** entra no catálogo de entregáveis como tipo próprio, e a DoD ganha um **bloco obrigatório para ações irreversíveis** (§7): backup/snapshot, dry-run + staging, estratégia de rollback no runbook, aprovação humana explícita.

3. **Uma única ferramenta SDD (Spec Kit)**. OpenSpec avaliado e **descartado**; a ideia útil (modelo delta) foi absorvida como a raia leve dentro do próprio fluxo.

Não adotado (YAGNI): papel "Plataforma/DevOps" nomeado separadamente — se é a mesma pessoa, nomear a *responsabilidade* basta.

Consequências

- Mudanças pequenas deixam de pagar o pedágio da spec completa, sem afrouxar verificação (revisão independente e testes continuam obrigatórios).
- Infra passa a ter rigor proporcional ao risco: a reversibilidade vira requisito de DoD, não boa intenção.
- Uma ferramenta só reduz carga cognitiva e de manutenção; a decisão sobre OpenSpec fica registrada para não ser reaberta sem novo contexto.
- `modelo-operacional.md` sobe para **v1.1.0** (MINOR); disciplina de branch/remote do FlowBuilder foi avaliada e **não** trazida (específica daquele repositório).

Fontes

- OpenSpec (Fission-AI): <https://github.com/Fission-AI/OpenSpec>
- Spec Kit vs OpenSpec: <https://intent-driven.dev/knowledge/spec-kit-vs-openspec/>
- Demais fontes da jornada em `docs/research/resultado-pesquisa-praticas-desenvolvimento-avaliacao.md`.

ADR 0006 — Enforcement da Definition of Done e forcing function do CHANGELOG

- **Status:** Aceito
- **Data:** 2026-07-22
- **Relacionado:** `docs/governance/modelo-operacional.md` (v1.2.0, §7, §12); ADR 0004/0005; Constituição v1.4.0 (Princípio V)

Contexto

O modelo operacional definia a Definition of Done como checklist verificável, mas a **garantia de aplicação** ainda dependia de disciplina. O princípio do elemento [8] diz que a garantia vem de **tornar os checks executáveis e bloqueantes** — aplicá-lo ao próprio modelo. O CI existente (`ci.yml`) já cobria testes, fitness functions (`architecture.test.ts` no web, `lint-imports` no api) e build/typecheck; faltavam a superfície de checklist humano/agente e a forcing function do CHANGELOG.

Decisão

Fechar o enforcement em quatro peças, dividindo mecânico (hard gate) de julgamento (checklist + aprovação humana):

1. **PR template** (`.github/pull_request_template.md`) espelhando a DoD (§7): raia, rastreabilidade, DoD, bloco de ações irreversíveis e gate de risco.
2. **Gate de CHANGELOG na CI** (`ci.yml` , job `changelog`): a PR falha se o `CHANGELOG.md` não for alterado; bypass via label `skip-changelog` . Cria-se o `CHANGELOG.md` no formato Keep a Changelog.
3. **Comando** `/dod` (`.claude/commands/dod.md`): self-check do agente que roda os checks locais e exige evidência antes de "pronto".
4. **Seção §12 "Aplicação e enforcement"** no modelo, mapeando cada item da DoD ao seu mecanismo (hard gate CI · setting nativo · checklist PR · comando · templates spec-kit).

Secret scanning fica como **setting nativo do GitHub** (secret scanning + push protection), não como job de CI — evita ferramenta e falso positivo (YAGNI).

Não adotado agora: **lint** (ruff/eslint) — não há tooling configurado; adicioná-lo falharia sobre código existente. Fica como follow-up (a DoD mantém "lint" como item de revisão até o tooling existir).

Consequências

- O que é mecânico bloqueia o merge sozinho (CI); o que exige julgamento fica no checklist do PR + aprovação humana ([9]) — a mesma divisão R/C→máquina, A→humano do [5] .
- O CHANGELOG deixa de depender de memória: quebra se esquecido.
- `modelo-operacional.md` sobe para **v1.2.0** (novo §12).
- Follow-up: habilitar o secret scanning nativo no repositório; avaliar lint (ruff/eslint).

Fontes

- Claude Code — *Best practices* (dê ao agente um check que ele rode):
<https://code.claude.com/docs/en/best-practices>
- Keep a Changelog: <https://keepachangelog.com/pt-BR/1.1.0/>
- `docs/handbook/09-definition-of-ready-done.md` ; `docs/governance/modelo-operacional.md` §12.

ADR 0007 — Separação do Maestro em repositório próprio

- **Status:** Aceito
- **Data:** 2026-07-22
- **Relacionado:** docs/governance/principios-maestro.md ; ADR 0004/0005/0006

Contexto

A metodologia (papéis, cerimônias, artefatos, raias, gates, DoD, governança) nasceu e amadureceu dentro de `ghdaru` (e antes, práticas de `flowbuilder`). Ela é **transversal a produto** — serve qualquer projeto humano+IA — e passou a ter massa própria: um modelo operacional versionado, um handbook de 12 capítulos, ADRs, templates e enforcement. Mantê-la acoplada ao repositório de uma plataforma específica limita sua evolução e reuso.

Decisão

Extrair a metodologia para o repositório próprio `GHDaru/maestro`, que passa a ser seu **lar único**, evoluindo de forma independente de `ghdaru` e `flowbuilder` (que ficam como consulta de origem, somente leitura).

Migrados (preservando o layout `docs/` para manter referências internas):

- `docs/governance/modelo-operacional.md` + novo `docs/governance/principios-maestro.md` (constituição própria da metodologia);
- `docs/handbook/` (12 capítulos + apresentações + prompts);
- ADRs 0004-0006 (números preservados);
- `docs/research/` (pesquisa + diário de aprendizado);
- templates: `.github/pull_request_template.md`, `.claude/commands/dod.md`, gate de CHANGELOG no CI.

Não migrados: a constituição da plataforma `ghdaru` e ADRs 0001-0003 (específicos de produto). No Maestro, "a Constituição" corresponde a `principios-maestro.md`.

Consequências

- Maestro evolui com versão própria; `ghdaru` / `flowbuilder` consomem a metodologia como referência.
- As cópias em `ghdaru` tornam-se redundantes; podem ser removidas de lá quando esta migração for confirmada (fora do escopo deste ADR).
- **Follow-up:** rebaixar as referências "Constituição / Princípio IV/V/VII" nos documentos migrados para apontar a `principios-maestro.md` (o mapa de linhagem está no fim daquele documento).

Fontes

- `docs/governance/principios-maestro.md` (nota de linhagem); `README.md`.